

Assignment 03: Rasters and Tables

For this assignment, you will be using Numpy, GDAL, and Pandas. Most of these questions can be solved using methods shown in lecture. However, some cases will require you to search the internet for answers. This is intended because efficiently searching documentation or Stackoverflow is a requirement of modern programming. This assignment also requires you to obtain data (I will tell you where) and organize it correctly so that you can access it via relative paths (do not use absolute paths anywhere in this assignment).

To turn this in, create a PDF of your finished Jupyter notebook using Quarto (we will discuss this on Tuesday) and upload it to Canvas.

Question 1

For this question, you will be using numpy to manipulate arrays in a way that works well for big data and will be using spatial tools to assess land-use change. Specifically, you will use data from a global, 300m resolution land-use, land-cover (LULC) map produced by ESACCI. They provide a wonderful time series of data from 1992-2019. If you want more information, check out their web viewer at <https://maps.elie.ucl.ac.be/CCI/viewer/>. Here you can view the data but also learn more about the classification regressions that uses time-series spectral data patterns to classify grid-cells into different types. I have extracted lulc maps for the country of Rwanda, which you will find in this assignment's data folder.

Part a.

Using the gdal package and the `gdal.Open()` function, open up the land-use, land-cover map for Rwanda in 2000.

In this file, there is only 1 band in this file, so you can also access it with the `GetRasterBand(1)` function. Without reading the whole array, show how you can determine how many total grid-cells there are in this country.

```
# 1A Answer
from osgeo import gdal
import numpy as np
import os
import matplotlib.pyplot as plt
import warnings
```

```
warnings.filterwarnings('ignore')

geotiff_filename_2000 = 'rwanda_lulc_2000.tif'
data_directory = '.././base_data'
geotiff_file_path_2000 = os.path.join(data_directory, geotiff_filename_2000)

rwanda_lulc_2000 = gdal.Open(geotiff_file_path_2000)

n_rows = rwanda_lulc_2000.RasterYSize
n_cols = rwanda_lulc_2000.RasterXSize
total_grid_cells = n_rows*n_cols

print('Total number of cells in this LULC map is: ', total_grid_cells)
```

Total number of cells in this LULC map is: 466284

Part b.

Using the results of part a, read the whole array into memory as a numpy array (the default option when using the ReadAsArray() function), and plot it using the matplotlib imshow command. Add a nice title to the plot describing what it is.

```
#1B Answer
array_2000 = rwanda_lulc_2000.ReadAsArray()

# First we find what LULC classes there are
classes = np.unique(array_2000)
print(classes)
```

```
[ 10  11  12  20  30  40  50  60  61  62  70 100 110 120 122 130 150 160
 170 180 190 200 210 255]
```

```
#Now we do the plotting

fig = plt.figure(figsize=(8, 6))
fig.set_dpi(300)

ax = fig.add_subplot()

ax.set_title('LULC Map in Rwanda, 2000')

# Using the ax we created, we call the imshow function on our array from earlier.
# This create a new "im" object
```

```

im = ax.imshow(array_2000)

# To fix the outlier problem from before, we use the im object to set its limits.
im.set_clim(10, 255)

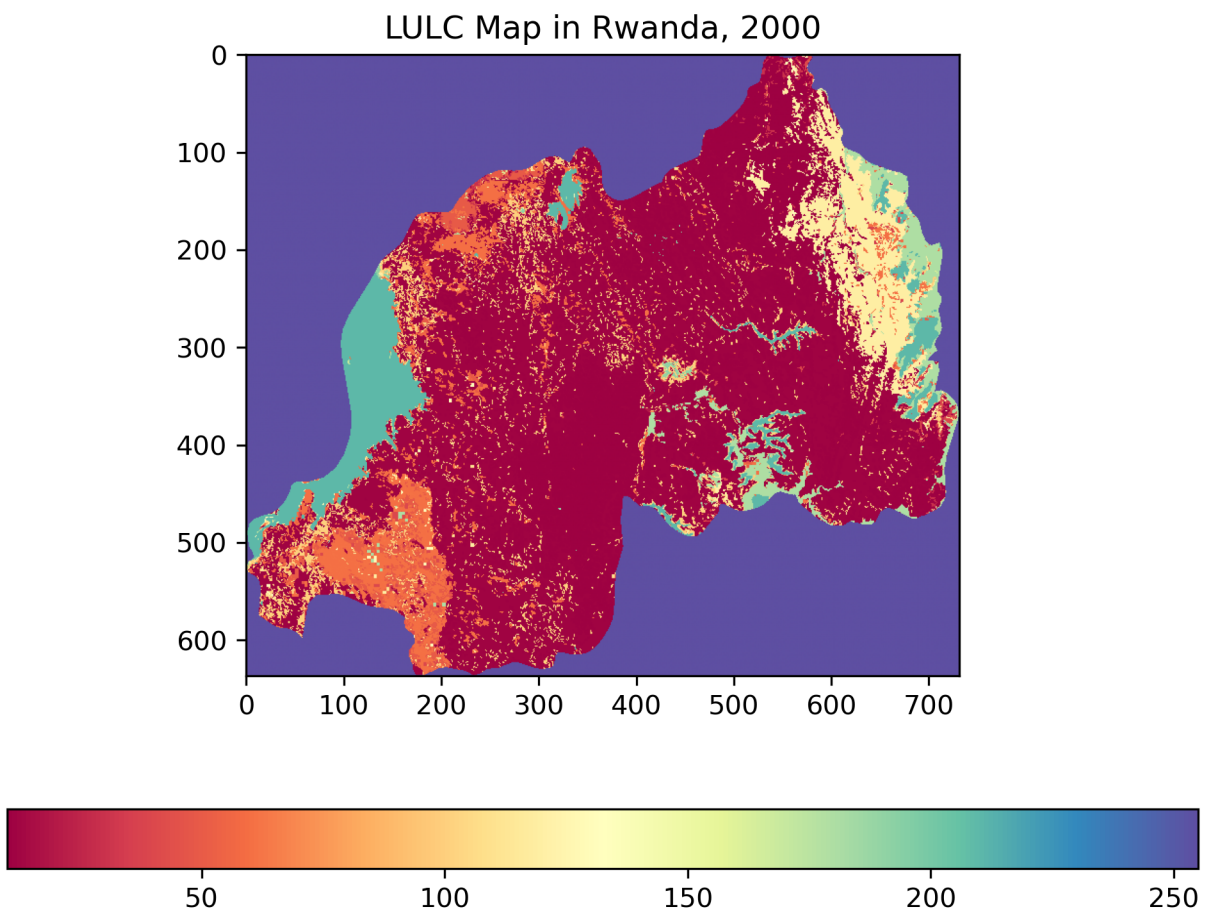
# Set the cbar to spectral
im.set_cmap('Spectral')

# Get the cmap
cmap = im.get_cmap()
cmap.set_over('white', alpha=1.0)

# Add the colorbar to the figure. It will generate its values from the im object.
fig.colorbar(im, orientation='horizontal')

plt.show()

```



Part c.

Using the legend you find at https://maps.elie.ucl.ac.be/CCI/viewer/download/ESACCI-LC-QuickUserGuide-LC-Maps_v2-0-7.pdf reclassify the LULC into a simplified map where 1 = cropland (including any mosaic types that are partially cropland) and 0 = anything else. Plot this using `imshow`.

```
# 1C Answer
from matplotlib.colors import ListedColormap
crop_2000 = np.isin(array_2000, [10, 20, 30, 40]).astype(np.uint8)

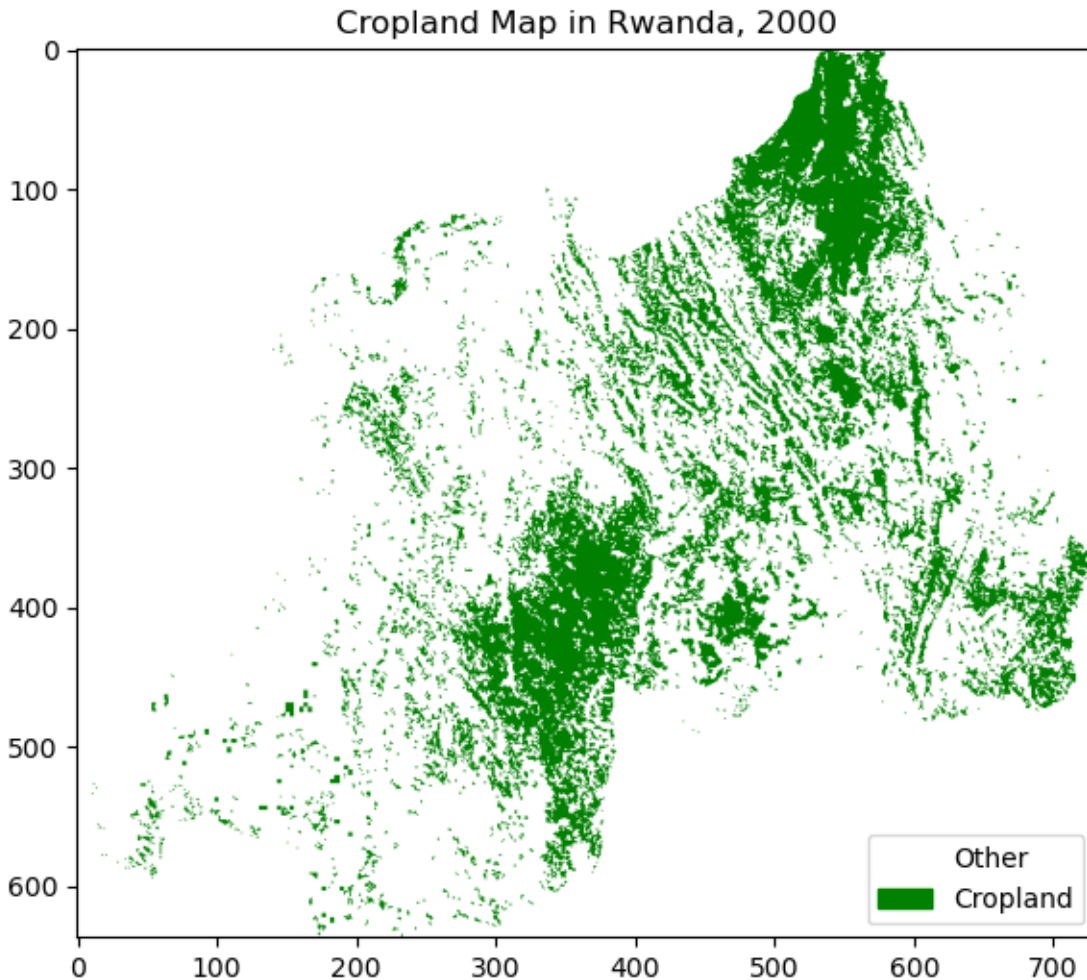
# Define colors: 0 = light gray, 1 = red (or whatever you want)
cmap = ListedColormap(['white', 'green'])

plt.figure(figsize=(8, 6))
im = plt.imshow(crop_2000, cmap=cmap)

# Create a custom legend
import matplotlib.patches as mpatches
legend_patches = [
    mpatches.Patch(color='white', label='Other'),
    mpatches.Patch(color='green', label='Cropland')
]

plt.legend(handles=legend_patches, loc='lower right')
plt.title("Cropland Map in Rwanda, 2000")
#plt.axis('off')

plt.show()
```



Part d.

Repeat the process for the 2010 LULC map. Using this array with the one from part c, create a new array that records where there was cropland expansion (i.e., there is cropland in 2010 but not in 2000) and where there was cropland abandonment (cropland in 2000 but not in 2010). Save this classification in a single new array. Plot this last array. Optionally, add a legend indicating which values in the array denote expansion and abandonment using some variant of 'ax.legend()'.

```
# 2D Answer
geotiff_filename_2010 = 'rwanda_lulc_2010.tif'
data_directory = '.././base_data'
geotiff_file_path_2010 = os.path.join(data_directory, geotiff_filename_2010)
rwanda_lulc_2010 = gdal.Open(geotiff_file_path_2010)
array_2010 = rwanda_lulc_2010.ReadAsArray()

cropland_codes = [10, 20, 30, 40]

crop_2010 = np.isin(array_2010, cropland_codes)
```

```

classification_2000_2010 = np.zeros_like(array_2000, dtype=np.uint8)

# expansion: in 2010, not in 2000
classification_2000_2010[(crop_2010 == True) & (crop_2000 == False)] = 1

# abandonment: in 2000, not in 2010
classification_2000_2010[(crop_2010 == False) & (crop_2000 == True)] = 2

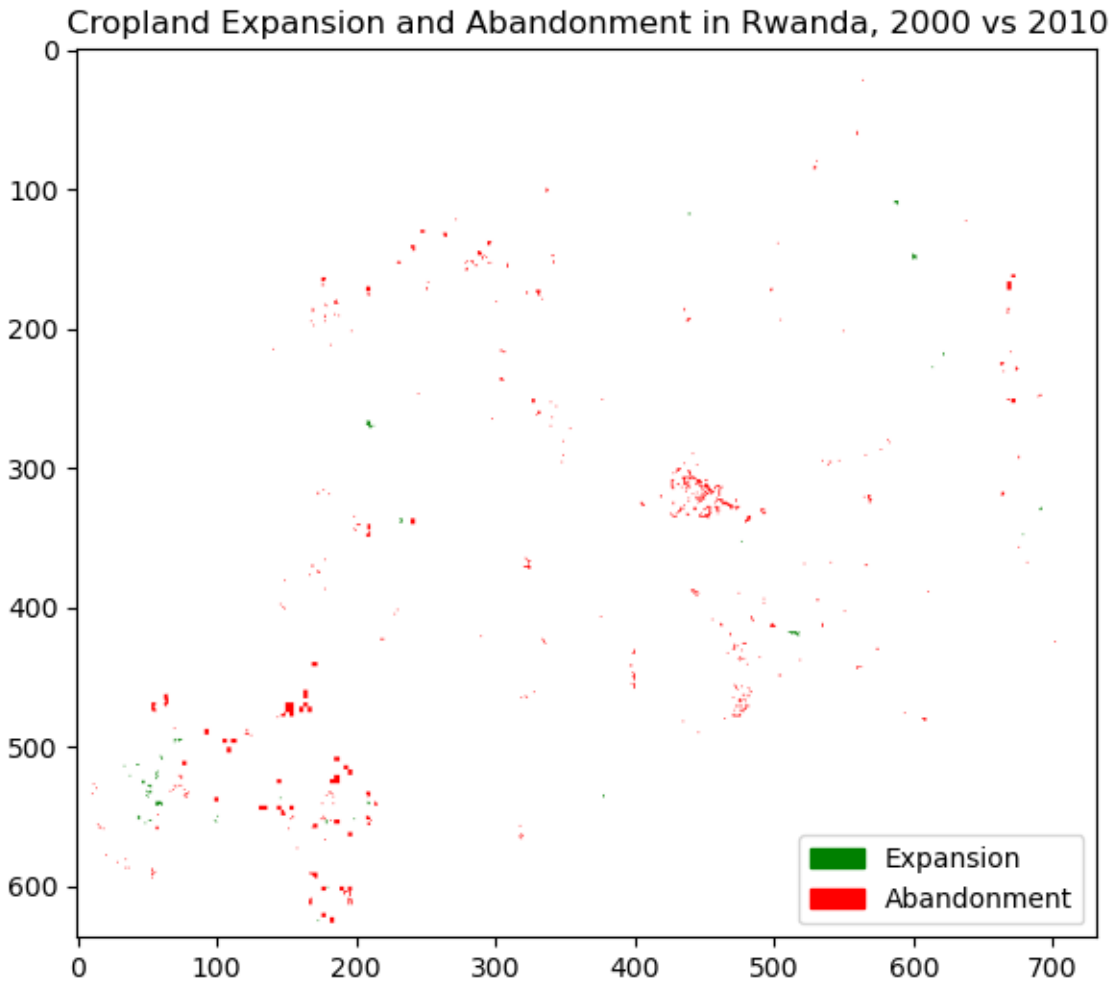
from matplotlib.colors import ListedColormap

# 0 = no change (white), 1 = expansion (green), 2 = abandonment (red)
cmap = ListedColormap(["white", "green", "red"])

plt.figure(figsize=(8, 6))
im = plt.imshow(classification_2000_2010, cmap=cmap, vmin=0, vmax=2)
plt.title("Cropland Expansion and Abandonment in Rwanda, 2000 vs 2010")

exp_patch = mpatches.Patch(color='green', label='Expansion')
aban_patch = mpatches.Patch(color='red', label='Abandonment')
plt.legend(handles=[exp_patch, aban_patch], loc='lower right')
plt.show()

```



Part e.

Repeat all the steps in part d, but with the 2015 LULC map, calculating the difference between 2015 and 2010.

```
# 2E Answer
```

```
geotiff_filename_2015 = 'rwanda_lulc_2015.tif'
data_directory = '.././base_data'
geotiff_file_path_2015 = os.path.join(data_directory, geotiff_filename_2015)
rwanda_lulc_2015 = gdal.Open(geotiff_file_path_2015)
array_2015 = rwanda_lulc_2015.ReadAsArray()

cropland_codes = [10, 20, 30, 40]

crop_2015 = np.isin(array_2015, cropland_codes)

classification_2010_2015 = np.zeros_like(array_2015, dtype=np.uint8)
```

```

# expansion: in 2015, not in 2010
classification_2010_2015[(crop_2015 == True) & (crop_2010 == False)] = 1

# abandonment: in 2000, not in 2010
classification_2010_2015[(crop_2015 == False) & (crop_2010 == True)] = 2

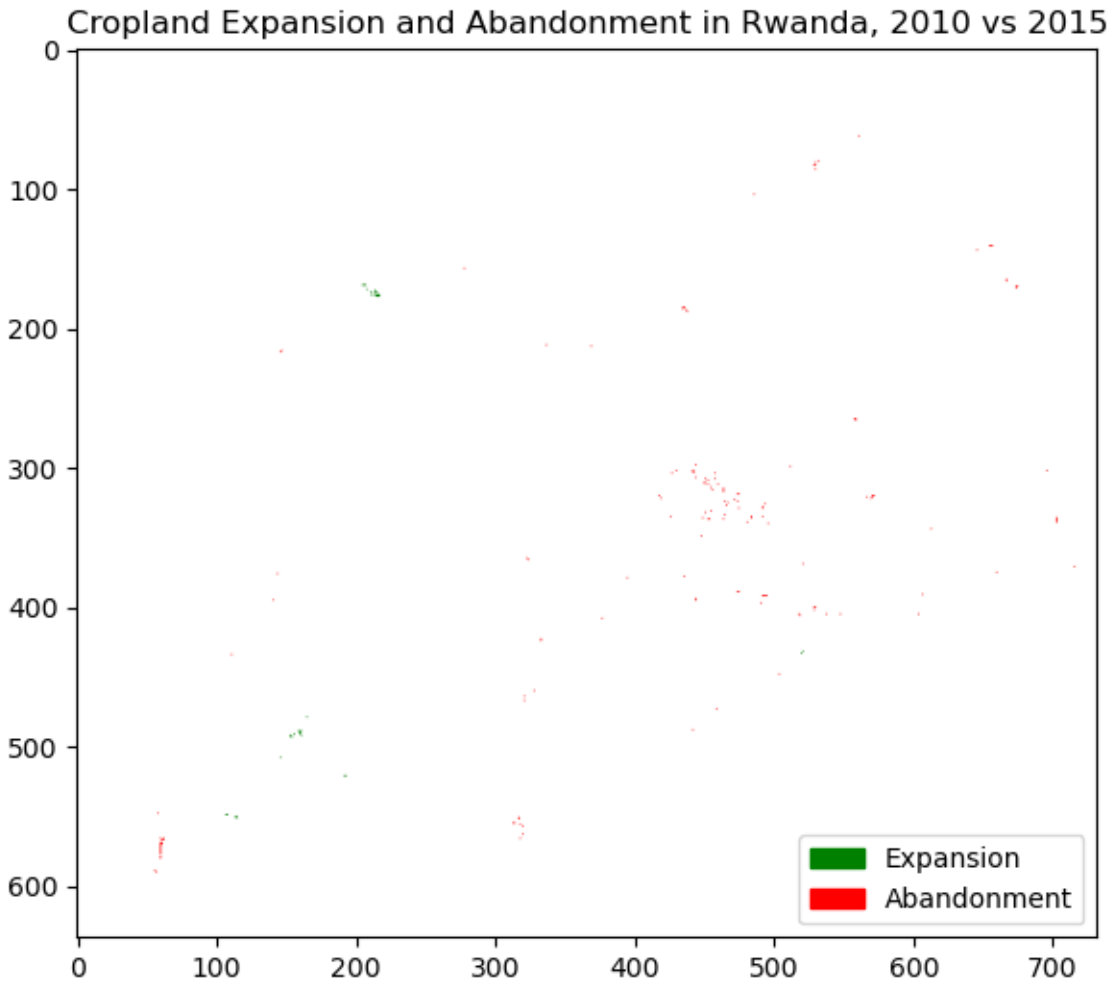
from matplotlib.colors import ListedColormap

# 0 = no change (white), 1 = expansion (green), 2 = abandonment (red)
cmap = ListedColormap(["white", "green", "red"])

plt.figure(figsize=(8, 6))
im = plt.imshow(classification_2010_2015, cmap=cmap, vmin=0, vmax=2)
plt.title("Cropland Expansion and Abandonment in Rwanda, 2010 vs 2015")

exp_patch = mpatches.Patch(color='green', label='Expansion')
aban_patch = mpatches.Patch(color='red', label='Abandonment')
plt.legend(handles=[exp_patch, aban_patch], loc='lower right')
plt.show()

```



Part f.

Use the two cropland binary rasters you processed above and make a new raster as follows:

0 = Never cropland

1 = Cropland in 2000 only

2 = Cropland in 2010 only

3 = Cropland in 2015 only

4 = Cropland in 2000 and 2010

5 = Cropland in 2000 and 2015

6 = Cropland in 2010 and 2015

7 = Cropland in 2000, 2010, and 2015.

Plot this raster using `imshow`. Add a legend indicating what each value means.

A tip for you: when you are combining multiple logical statements in numpy, each individual logical statement must be surrounded by parentheses. Additionally, you have to use the bitwise logical operators `&` and `|` instead of the normal logical operators `and` and `or` (numpy requires this for specificity).

For example, `np.where((a == 1) & (b == 2))` will return the indices where `a` is 1 and `b` is 2. If you do `np.where(a == 1 & b == 2)` or `np.where((a == 1) and (b == 2))`, you will get an error.

```
#2F Answer

classification = np.zeros_like(crop_2000, dtype=np.uint8)

# 1 = 2000 only
classification[(crop_2000 == True) &
                (crop_2010 == False) &
                (crop_2015 == False)] = 1

# 2 = 2010 only
classification[(crop_2000 == False) &
                (crop_2010 == True) &
                (crop_2015 == False)] = 2

# 3 = 2015 only
classification[(crop_2000 == False) &
                (crop_2010 == False) &
                (crop_2015 == True)] = 3

# 4 = 2000 & 2010
classification[(crop_2000 == True) &
                (crop_2010 == True) &
                (crop_2015 == False)] = 4

# 5 = 2000 & 2015
classification[(crop_2000 == True) &
                (crop_2010 == False) &
                (crop_2015 == True)] = 5

# 6 = 2010 & 2015
classification[(crop_2000 == False) &
                (crop_2010 == True) &
                (crop_2015 == True)] = 6

# 7 = 2000 & 2010 & 2015
classification[(crop_2000 == True) &
                (crop_2010 == True) &
                (crop_2015 == True)] = 7

import matplotlib.pyplot as plt
```

```

from matplotlib.colors import ListedColormap
import matplotlib.patches as mpatches

cmap = ListedColormap([
    "white",      # 0 never cropland
    "blue",       # 1 2000 only
    "orange",     # 2 2010 only
    "gold",       # 3 2015 only
    "brown",      # 4 2000 & 2010
    "pink",       # 5 2000 & 2015
    "purple",     # 6 2010 & 2015
    "green"       # 7 2000, 2010, 2015
])

plt.figure(figsize=(8, 6))
im = plt.imshow(classification, cmap=cmap, vmin=0, vmax=7)
plt.title("Cropland Changes in Rwanda in Years 2000, 2010 and 2015")
plt.axis('off')

legend_labels = {
    0: "0 = Never cropland",
    1: "1 = Cropland in 2000 only",
    2: "2 = Cropland in 2010 only",
    3: "3 = Cropland in 2015 only",
    4: "4 = Cropland in 2000 & 2010",
    5: "5 = Cropland in 2000 & 2015",
    6: "6 = Cropland in 2010 & 2015",
    7: "7 = Cropland in 2000, 2010, 2015"
}

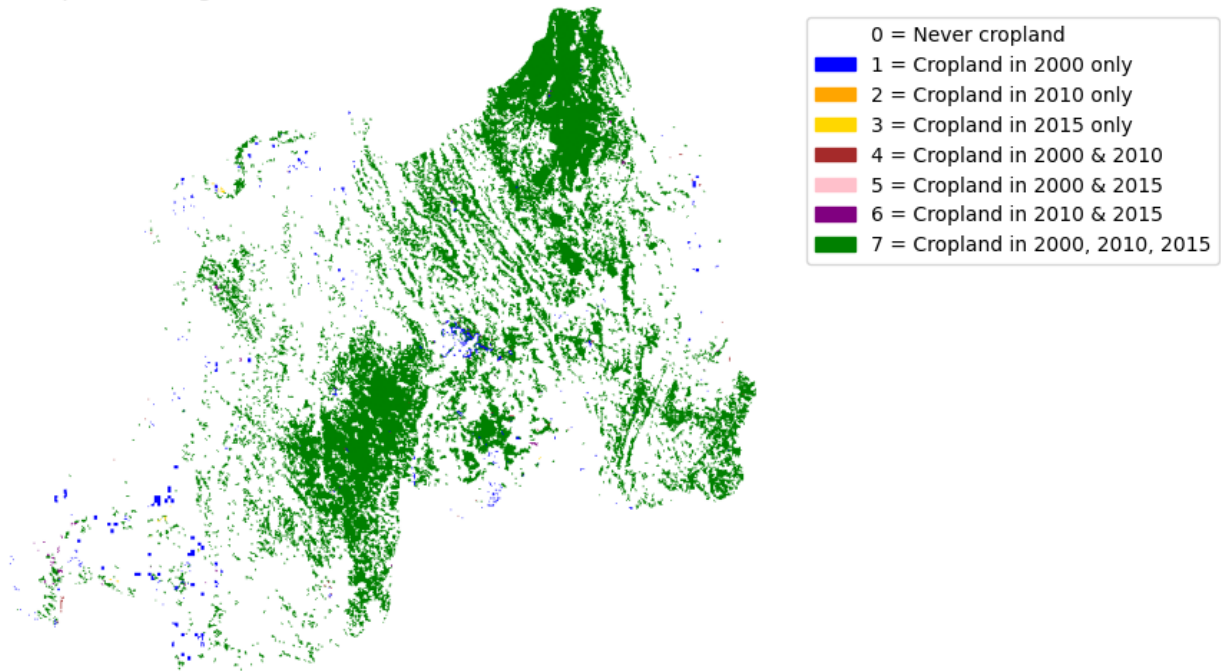
patches = [
    mpatches.Patch(color=cmap(i), label=legend_labels[i])
    for i in range(8)
]

plt.legend(
    handles=patches,
    bbox_to_anchor=(1.05, 1),
    loc='upper left'
)

plt.show()

```

Cropland Changes in Rwanda in Years 2000, 2010 and 2015



Part g.

Describe your results. What do you notice about the spatial patterns of cropland expansion and abandonment? What do you think might be driving these patterns? Were they clustered in any specific area? Were the changes the same between 2000-2010 and 2010-2015?

Answer: Most of the cropland was untouched in Rwanda over 2000-2015 (green areas, where cropland was present in 2000, 2010 and 2015). We can see some blue patches overlaid against the green areas: these are lands that were croplands in 2000 and later were abandoned and not reclaimed for crop production (at least not as of 2015). These blue areas (where cropland was present only in 2000) are concentrated in the southwestern parts of the country, as well as some central areas.

The southwestern parts of the country, where cropland was only present in 2000, are close to Lake Kivu; I assume that these were cropland close to a large water reservoir for irrigation (the lake) that was abandoned for urban development. The central areas where blue clusters are present are around the capital of the country, Kigali. In a similar fashion, I assume that these were lands that were previously used for growing crops and were abandoned for urban development near the capital city of Rwanda, Kigali.

The changes were not the same between 2000-2010 and 2010-2015. On the map, the most prominent areas indicating change are the blue ones, which means that cropland was abandoned between 2000 and 2010, and not reclaimed between 2010 and 2015. Colors that would indicate changes between 2010 and 2015 specifically (brown, code 4, or orange, code 2) are not prominent on this picture, which means that little change took place between 2010 and 2015, compared to time between 2000 and 2010.

Question 2

For this question, you will analyze crops using two datasets:

1. FAOSTAT Production_Crops_E_All_Data_(Normalized).csv. You could download it yourself but please use the one in the Class's Data Directory.
2. MAPSPAM (Spatial Production Allocation Mode). You can find the download listing here: <https://dataverse.harvard.edu/dataset.xhtml?persistentId=doi:10.7910/DVN/PRFF8V> Download the readme and all of the files that end with .geotiff.zip. Unzip them and put them in a folder called spam in your Class's Data Directory. This is a big download, so it may take a while. Go watch a show or something. I don't know, what do people do nowadays for fun?3.

Sidenote: MAPSPAM was a competitor to the EARTHSTAT data we showed in class before. However, MAPSPAM continues to be updated and is more open-source, so I am in the process of switching my whole GTAP-InVEST model to it.

You will choose 5 crops that interest you, making sure to select crops that are in both MAPSPAM and FAOSTAT dataset. You will want to refer to the ReadMe_v2r0_Global.txt file, which was available at the same download site for MAPSPAM, to identify what each of the different crops mean. For each crop you choose (use a loop), calculate the global sum of production tonnage for irrigated crops, all rainfed crops combined but excluding subsistence crops, and for all crops together. Report those values via print().

Next, for the same crops, compare how the value you calculated above compares to the value in the FAOSTAT dataset. To do this, you will need to use Pandas to load the CSV and then figure out how to get just the production tonnage values for the crops you are interested in. Interpret your results for your crops. Is FAO higher/lower than MAPSPAM? Do you notice any patterns to the difference?

```
# Question 2 answer
# I (irrigated), R-S (rainfed without subsistence), A (all)

# Path to the folder with .tif files
data_folder = "../../base_data/spam"

# Crops and technology categories
crop_names = {"BARL": "Barley",
              "POTA": "Potatoes",
              "TEAS": "Tea",
              "CNUT": "Coconuts",
              "SUGC": "Sugar cane"}
tech_names = {"A": "All",
              "I": "Irrigated",
              "R": "Rainfed (without subsistence)"} # 'R' = Rainfed minus subsistence

# Initialize dictionary to store results
global_production = {crop: {"A": 0, "I": 0, "R": 0} for crop in crop_names}

# Helper function to read a raster safely
```

```

def read_raster_clean(path):
    """
    This function takes in a crop production raster file and returns a numpy array.
    Sets any missing values to 0.
    Small negative estimation residuals are also set to 0.
    """
    ds = gdal.Open(path)
    band = ds.GetRasterBand(1)

    nodata = band.GetNoDataValue()
    arr = band.ReadAsArray().astype(float)

    # Mask nodata values
    if nodata is not None:
        arr[arr == nodata] = np.nan

    # Mask any negative values (SPAM sometimes has small negative estimation residuals)
    arr[arr < 0] = np.nan
    ds = None
    return arr # return dataset so caller can close it

# Loop over crops
for crop in crop_names:
    for tech in tech_names:
        if tech == "R":
            # Rainfed minus subsistence
            file_R = os.path.join(data_folder, f"spam2010V2r0_global_P_{crop}_R.tif")
            file_S = os.path.join(data_folder, f"spam2010V2r0_global_P_{crop}_S.tif")

            arr_R = read_raster_clean(file_R)
            arr_S = read_raster_clean(file_S)

            arr_diff = arr_R - arr_S

            # Sum all values ignoring nan
            global_production[crop]["R"] = np.nansum(arr_diff)

        else:
            # All (A) or Irrigated (I)
            file_path = os.path.join(data_folder, f"spam2010V2r0_global_P_{crop}_{tech}.tif")
            arr = read_raster_clean(file_path)
            global_production[crop][tech] = np.nansum(arr)

# Print results
print("Worldwide production of selected crops from MAPSPAM:\n")
for crop_code, values in global_production.items():
    print(f"{crop_names[crop_code]}:")

```

```
for tech_code, prod in values.items():
    print(f" {tech_names[tech_code]}: {prod:.2f} tons")
```

Worldwide production of selected crops from MAPSPAM:

Barley:

All: 135892346.81 tons
 Irrigated: 14687636.31 tons
 Rainfed (without subsistence): 116798486.29 tons

Potatoes:

All: 346402155.90 tons
 Irrigated: 80164252.59 tons
 Rainfed (without subsistence): 229317409.45 tons

Tea:

All: 4553133.01 tons
 Irrigated: 184293.40 tons
 Rainfed (without subsistence): 4139420.01 tons

Coconuts:

All: 59558063.38 tons
 Irrigated: 650618.70 tons
 Rainfed (without subsistence): 46947528.07 tons

Sugar cane:

All: 1720477083.60 tons
 Irrigated: 719246012.90 tons
 Rainfed (without subsistence): 982314324.97 tons

```
import pandas as pd

file_path = "../../../base_data/spam/Production_Crops_E_All_Data_(Normalized).csv"

# Load CSV into a DataFrame
df = pd.read_csv(file_path, encoding="latin1")

selected_crops = ['Barley', 'Potatoes', 'Tea', 'Sugar cane', 'Coconuts']
df_filtered = df[
    (df['Item'].isin(selected_crops)) &    # Crop filter
    (df['Element'] == 'Production') &    # Element filter
    (df['Year'] == 2010)                  # Year filter
]

production_by_crop = df_filtered.groupby('Item')['Value'].sum()

print("Worldwide production of selected crops from FAOSTAT:\n")
for crop, value in production_by_crop.items():
    print(f"{crop}: {value:.2f} tons")
```

Worldwide production of selected crops from FAOSTAT:

Barley: 566391692.00 tons
Coconuts: 269931132.00 tons
Potatoes: 1592197392.00 tons
Sugar cane: 7509928810.00 tons
Tea: 23650666.00 tons

The aggregate production data in 2010 we get from MAPSPAM by summing across the aggregate production rasters is smaller than the data we get from FAOSTAT. For Barley, FAOSTAT gives 566,391,692.00 tons, whereas MAPSPAM gives 135,892,346.81 tons, about 4.2 times smaller. For Coconuts, FAOSTAT gives 269,931,132.00 tons, whereas MAPSPAM gives 59,558,063.38, about 4.5 times smaller. For Potatoes, FAOSTAT gives 1,592,197,392.00 tons, whereas MAPSPAM gives 346,402,155.90 tons, about 4.6 times smaller. For Sugar Cane, FAOSTAT gives 7,509,928,810.00 tons, whereas MAPSPAM 1,720,477,083.60, about 4.4 times smaller. Finally, for Tea, FAOSTAT gives 23,650,666.00 tons, whereas MAPSPAM gives 4,553,133.01 tons, about 5.2 times smaller. Overall, we can notice a pattern: production figures from MAPSPAM data are about 4-5 times smaller than the ones obtained from FAOSTAT for all 5 selected crops.